



JavaScript Modules & Module Systems

Topics: ES Modules, Named/Default Export, Dynamic Import, CommonJS, ESM vs CommonJS

1. Module কী?

একটা বড় JavaScript application-কে ছোট ছোট, manageable file-এ ভাগ করাকে **module system** বলা যায়।

ধরো তোমার project:

```
src/  
├─ auth.js  
├─ user.js  
├─ cart.js  
├─ payment.js  
└─ app.js
```

প্রতিটি file নিজের কিছু functionality রাখবে এবং প্রয়োজন হলে অন্য file থেকে functionality `import` করবে।

Module ব্যবহার করার মূল কারণ

- Code organization
- Reusability
- Maintainability
- Encapsulation
- Dependency management
- Large application manage করা সহজ
- Global namespace pollution কমানো

2. Module ছাড়া সমস্যা কী?

ধরো একটা project-এ সব code এক file-এ:

```
const userName = "Sharlar";  
  
function login() {  
  // ...  
}
```

```
function logout() {  
  // ...  
}  
  
function calculateTotal() {  
  // ...  
}  
  
function payment() {  
  // ...  
}
```

Project বড় হলে:

```
10 lines  
100 lines  
1000 lines  
5000 lines  
10000+ lines
```

একটা file-এর মধ্যে সব functionality রাখা difficult হয়ে যায়।

তাই আমরা আলাদা করি:

```
auth.js  
cart.js  
payment.js  
user.js
```

তারপর প্রয়োজন অনুযায়ী import করি।

3. JavaScript-এর প্রধান Module Systems

JavaScript ecosystem-এ মূলত দুই ধরনের module system সবচেয়ে গুরুত্বপূর্ণ:

Module System	Syntax
ES Modules	<code>import</code> / <code>export</code>
CommonJS	<code>require()</code> / <code>module.exports</code>

ES Modules

```
import { add } from "./math.js";
```

CommonJS

```
const { add } = require("./math");
```

Modern JavaScript development-এ **ES Modules (ESM)** সবচেয়ে important ।

4. ES Modules (ESM)

ES Modules হলো JavaScript-এর standardized module system ।

প্রধান syntax:

```
export  
import
```

Example:

math.js

```
export function add(a, b) {  
  return a + b;  
}
```

app.js

```
import { add } from "./math.js";  
  
console.log(add(10, 20));
```

এখানে:

```
math.js  
↓  
exports add  
↓
```

```
app.js
↓
imports add
```

5. export

export ব্যবহার করে কোনো variable, function, class ইত্যাদি অন্য module-এর জন্য available করা যায়।

5.1 Named Export

```
export const name = "Sharlar";

export function greet() {
  return "Hello";
}
```

এগুলোকে **named exports** বলা হয়।

Import করার সময় একই নাম ব্যবহার করতে হয়:

```
import { name, greet } from "./user.js";
```

6. Named Export-এর Example

math.js

```
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

app.js

```
import { PI, add, subtract } from "./math.js";

console.log(PI);
console.log(add(10, 5));
console.log(subtract(10, 5));
```

7. Named Export আলাদা করে শেষে করা

চাইলে declaration-এর সময় `export` না লিখে শেষে `export` করতে পারো।

```
const name = "Shariar";

function greet() {
  return "Hello";
}

export { name, greet };
```

এটাও valid।

8. Named Import-এর নাম পরিবর্তন করা

`as` ব্যবহার করে imported name পরিবর্তন করা যায়।

```
import { add as sum } from "./math.js";

console.log(sum(10, 20));
```

এখানে:

```
original name → add
local name    → sum
```

9. একাধিক Named Import

```
import {  
  add,  
  subtract,  
  multiply,  
  divide  
} from "./math.js";
```

10. সব Named Export একসাথে Import

* as ব্যবহার করা যায়।

math.js

```
export function add(a, b) {  
  return a + b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}
```

app.js

```
import * as math from "./math.js";  
  
console.log(math.add(10, 5));  
console.log(math.subtract(10, 5));
```

এখানে **math** একটি namespace object-এর মতো কাজ করছে।

11. Default Export

একটি module থেকে একটি **primary/default value** export করতে **export default** ব্যবহার করা যায়।

```
export default function greet() {  
  console.log("Hello");  
}
```

Import:

```
import greet from "./greet.js";  
  
greet();
```

12. Default Export-এর গুরুত্বপূর্ণ বৈশিষ্ট্য

Default export import করার সময় একই নাম ব্যবহার করা বাধ্যতামূলক নয়।

greet.js

```
export default function greet() {  
  console.log("Hello");  
}
```

এখন:

```
import greet from "./greet.js";
```

অথবা:

```
import sayHello from "./greet.js";
```

দুটোই valid।

কারণ এটা default export।

13. Named vs Default Export

এটা খুব important interview topic।

Feature	Named Export	Default Export
Syntax	<code>export</code>	<code>export default</code>
Import <code>{ }</code> লাগে?	Yes	No
Name match করতে হয়?	Yes	No
একটি module-এ কয়টি?	অনেকগুলো	সর্বোচ্চ একটি
Example	<code>export const x</code>	<code>export default x</code>

Named

```
export const age = 30;
```

```
import { age } from "./user.js";
```

Default

```
export default age;
```

```
import age from "./user.js";
```

14. একই Module-এ Named + Default Export

একই module-এ দুটোই থাকতে পারে।

```
export const name = "Shariar";

export function greet() {
  return "Hello";
}

export default function user() {
  return {
    name: "Shariar"
  };
}
```

Import:

```
import user, { name, greet } from "./user.js";
```

এখানে:

```
user → default export  
name → named export  
greet → named export
```

15. Default Export কেন ব্যবহার করব?

যখন কোনো module-এর একটি প্রধান responsibility থাকে।

Example:

```
Button.js  
User.js  
PaymentService.js  
Router.js
```

যেমন:

```
export default function Button() {  
  // ...  
}
```

React ecosystem-এ default export historically খুব common:

```
export default App;
```

16. Named Export কেন ব্যবহার করব?

যখন একটি module থেকে related multiple functionality export করতে চাই।

```
export function add() {}  
  
export function subtract() {}
```

```
export function multiply() {}  
  
export function divide() {}
```

তরপর:

```
import {  
  add,  
  subtract,  
  multiply,  
  divide  
} from "./math.js";
```

17. `import` Syntax Variations

Named import

```
import { add } from "./math.js";
```

Multiple named imports

```
import { add, subtract } from "./math.js";
```

Rename

```
import { add as sum } from "./math.js";
```

Default import

```
import math from "./math.js";
```

Default + named

```
import math, { add, subtract } from "./math.js";
```

Namespace import

```
import * as math from "./math.js";
```

18. Import শুধুমাত্র Side Effect-এর জন্য

কখনো কোনো module-এর exported value দরকার হয় না, কিন্তু module-এর code execute হওয়া দরকার।

```
import "./analytics.js";
```

এটাকে **side-effect import** বলা হয়।

Example:

```
import "./styles.css";
```

Frontend tooling-এ এই pattern খুব common।

19. Module Scope

Modules-এর নিজেদের আলাদা scope থাকে।

user.js

```
const username = "Shariar";
```

app.js

```
console.log(username);
```

এটা কাজ করবে না।

কারণ:

```
user.js scope
  ≠
app.js scope
```

Explicitly export/import করতে হবে।

20. Modules Global Scope Pollution কমায়ে

Normal script-এর ক্ষেত্রে global variable তৈরি হওয়ার ঝুঁকি বেশি।

Module-এ:

```
const username = "Shariar";
```

এই variable automatically অন্য module-এর কাছে available হয়ে যায় না।

এটা module system-এর বড় advantage।

21. Static Import

Normal `import` হলো **static import**।

```
import { add } from "./math.js";
```

এটা source code-এর structure-এর অংশ।

Bundler/tooling আগেই dependency graph তৈরি করতে পারে।

```
app.js
↓
math.js
↓
utils.js
```

22. Dynamic `import()`

Dynamic import ব্যবহার করে runtime-এ module load করা যায়।

Syntax:

```
import("./math.js");
```

এটা একটি **Promise** return করে।

Example:

```
const module = await import("./math.js");  
  
console.log(module);
```

23. Dynamic Import কেন দরকার?

সব code শুরুতেই load করার প্রয়োজন নেই।

ধরো application:

```
Home  
Dashboard  
Admin Panel  
Payment  
Reports  
Analytics
```

User যদি শুধু Home page visit করে, তাহলে Admin Panel-এর code শুরুতেই load করার দরকার নেই।

Dynamic import দিয়ে প্রয়োজনের সময় load করা যায়।

24. Dynamic Import Example

```
async function loadMath() {  
  const math = await import("./math.js");
```

```
    console.log(math.add(10, 20));  
  }  
  
  loadMath();
```

25. Dynamic Import Promise Return করে

```
const modulePromise = import("./math.js");  
  
console.log(modulePromise);
```

এটা Promise ।

তাই:

```
const module = await import("./math.js");
```

অথবা:

```
import("./math.js")  
  .then((module) => {  
    console.log(module);  
  })  
  .catch((error) => {  
    console.error(error);  
  });
```

26. Dynamic Import-এর Practical Use Case

Feature-based loading

```
async function openEditor() {  
  const { openEditor } = await import("./editor.js");  
  
  openEditor();  
}
```

User editor open করলে তখন module load হবে।

27. Lazy Loading

Lazy loading মানে কোনো resource প্রয়োজন হওয়ার আগ পর্যন্ত load না করা।

Traditional:

```
import Editor from './Editor.js';
```

এতে application startup-এর dependency graph-এর অংশ হিসেবে editor module load/bundle হতে পারে।

Lazy:

```
const Editor = await import('./Editor.js');
```

প্রয়োজনের সময় module load করা যায়।

28. Code Splitting

Modern bundlers dynamic imports দেখে application code-কে আলাদা chunks-এ ভাগ করতে পারে।

ধরো:

```
main.js  
dashboard.js  
admin.js  
editor.js
```

User শুধু dashboard-এ গেলে:

```
main.js  
dashboard.js
```

প্রথমে দরকার হতে পারে।

Admin code পরে load হতে পারে।

এটাকে **code splitting** বলা হয়।

29. Dynamic Import + Named Export

math.js

```
export function add(a, b) {  
  return a + b;  
}  
  
export function subtract(a, b) {  
  return a - b;  
}
```

app.js

```
const math = await import("./math.js");  
  
console.log(math.add(10, 5));
```

কারণ dynamic import পুরো module namespace object return করে।

30. Dynamic Import + Default Export

user.js

```
export default function getUser() {  
  return {  
    name: "Sharir"  
  };  
}
```

Import:

```
const module = await import("./user.js");  
  
const getUser = module.default;
```

```
console.log(getUser());
```

এখানে default export পাওয়া যায়:

```
module.default
```

31. CommonJS

CommonJS হলো JavaScript-এর একটি module system, যা Node.js ecosystem-এ historically খুব popular।

Main syntax:

```
require()  
module.exports  
exports
```

32. CommonJS Export

math.js

```
function add(a, b) {  
  return a + b;  
}  
  
module.exports = {  
  add  
};
```

Import:

```
const { add } = require("./math");  
  
console.log(add(10, 20));
```

33. CommonJS `module.exports`

একটি value export করা যায়:

```
module.exports = function greet() {  
  console.log("Hello");  
};
```

Import:

```
const greet = require("./greet");  
  
greet();
```

34. Multiple CommonJS Exports

```
function add(a, b) {  
  return a + b;  
}  
  
function subtract(a, b) {  
  return a - b;  
}  
  
module.exports = {  
  add,  
  subtract  
};
```

Import:

```
const {  
  add,  
  subtract  
} = require("./math");
```

35. exports কী?

CommonJS-এ:

```
exports.add = add;
```

এটা সাধারণত equivalent:

```
module.exports.add = add;
```

Example:

```
exports.add = function (a, b) {  
  return a + b;  
};
```

Import:

```
const { add } = require("./math");
```

36. exports বনাম module.exports

এটা খুব common interview question ।

Initially:

```
exports === module.exports
```

ধারণগতভাবে একই object reference থাকে ।

তাই:

```
exports.add = add;
```

কাজ করে ।

কিন্তু:

```
exports = add;
```

এটা module-এর actual exported value replace করে না।

37. Correct CommonJS Pattern

Multiple exports:

```
module.exports = {  
  add,  
  subtract  
};
```

অথবা:

```
exports.add = add;  
exports.subtract = subtract;
```

Simple rule:

যদি পুরো **exported value replace** করতে চাও, `module.exports` ব্যবহার করো।

38. CommonJS `require()`

```
const math = require("./math");
```

তারপর:

```
math.add(10, 20);
```

অথবা:

```
const { add } = require("./math");
```

39. CommonJS Module Execution

CommonJS module সাধারণত প্রথমবার `require()` হলে execute হয়।

তারপর Node.js module cache ব্যবহার করে।

Conceptually:

```
require("./math")
  ↓
Load module
  ↓
Execute module
  ↓
Cache result
  ↓
Return exports
```

পরবর্তীতে একই module require করলে cached result ব্যবহার হতে পারে।

40. ESM vs CommonJS

সবচেয়ে important comparison:

Feature	ES Modules	CommonJS
Export	<code>export</code>	<code>module.exports</code>
Import	<code>import</code>	<code>require()</code>
Default export	<code>export default</code>	Native ESM-style default নেই
Dynamic loading	<code>import()</code>	<code>require()</code> সাধারণত synchronous
Standard	JavaScript standard	Node.js ecosystem-এর traditional system
Browser support	Native	Native browser module system নয়
Static analysis	Strong	তুলনামূলকভাবে কম
Tree shaking	সহজতর	কঠিনতর
Typical extension	<code>.js</code> / <code>.mjs</code>	<code>.js</code> / <code>.cjs</code>
Modern frontend	Very common	Rare

Feature	ES Modules	CommonJS
Traditional Node.js	Less common historically	Very common

41. ESM vs CommonJS — সবচেয়ে সহজ Mental Model

ESM

```
export
import
```

CommonJS

```
module.exports
require()
```

মনে রাখো:

```
ESM      → import/export
CommonJS → require/module.exports
```

42. Browser-এ ES Modules

Browser সরাসরি ESM support করে।

HTML:

```
<script type="module" src="./app.js"></script>
```

math.js

```
export function add(a, b) {
  return a + b;
}
```

app.js

```
import { add } from "./math.js";  
  
console.log(add(10, 20));
```

43. type="module"

Browser-এ module script বলতে:

```
<script type="module" src="./app.js"></script>
```

type="module" না দিলে browser এটাকে normal script হিসেবে treat করবে।

44. Node.js-এ ESM Enable করা

Node.js project-এ package.json -এ:

```
{  
  "type": "module"  
}
```

দিলে .js files-কে ESM হিসেবে interpret করা যায়।

তারপর:

```
import { add } from "./math.js";
```

45. .mjs

.mjs extension explicitভাবে ES Module indicate করে।

```
math.mjs  
app.mjs
```

Example:

```
// math.mjs  
export function add(a, b) {  
  return a + b;  
}
```

```
// app.mjs  
import { add } from "./math.mjs";  
  
console.log(add(10, 20));
```

46. .cjs

`.cjs` extension CommonJS indicate করে।

```
math.cjs  
app.cjs
```

Example:

```
// math.cjs  
  
function add(a, b) {  
  return a + b;  
}  
  
module.exports = { add };
```

```
// app.cjs  
  
const { add } = require("./math.cjs");  
  
console.log(add(10, 20));
```

47. Node.js Module Type Mental Model

Node.js project-এ broadly:

```
"type": "module"
  ↓
.js → ESM
```

আর:

```
.cjs
```

হলে CommonJS।

এবং:

```
.mjs
```

হলে ESM।

48. Static Import বনাম Dynamic Import

	Static Import	Dynamic Import
Syntax	<code>import x from './x.js'</code>	<code>import('./x.js')</code>
Runtime decision	No	Yes
Returns Promise	No	Yes
Lazy loading	Suitable	Suitable
Code splitting	Bundlers can use	Very useful
Conditional loading	No	Yes

49. Conditional Dynamic Import

Dynamic import-এর সবচেয়ে useful feature হলো condition অনুযায়ী module load করা।

```
if (user.isAdmin) {  
  const admin = await import("./admin.js");  
  
  admin.openDashboard();  
}
```

Admin না হলে:

```
admin.js
```

load করার প্রয়োজন নেই।

50. Dynamic Import with Event

```
button.addEventListener("click", async () => {  
  const module = await import("./heavyFeature.js");  
  
  module.start();  
});
```

User click করার পর feature load হবে।

এটাই lazy-loading-এর practical pattern।

51. Modules এবং Tree Shaking

Modern bundler unused exports বাদ দিতে পারে।

Example:

```
export function add() {}  
export function subtract() {}  
export function multiply() {}  
export function divide() {}
```

যদি application শুধু:

```
import { add } from "./math.js";
```

ব্যবহার করে, bundler optimization করে unused code বাদ দিতে পারে।

এটাকে **tree shaking** বলা হয়।

ESM-এর static structure tree shaking-এর জন্য সুবিধাজনক।

52. Module Dependency Graph

ধরো:

```
app.js
├── user.js
│   └── api.js
├── cart.js
│   └── product.js
└── utils.js
```

Bundler এই dependency relationship analyze করতে পারে।

এটাই application-এর **module dependency graph**।

53. Circular Dependency

দুটি module একে অপরের উপর depend করলে circular dependency তৈরি হতে পারে।

```
A.js
↓
B.js
↓
A.js
```

Example:

A.js

```
import { b } from "./B.js";  
  
export const a = "A";
```

B.js

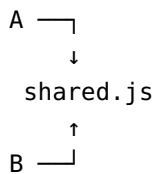
```
import { a } from "./A.js";  
  
export const b = "B";
```

এ ধরনের architecture confusing behavior তৈরি করতে পারে।

Best Practice

Circular dependency avoid করার চেষ্টা করো।

Common solution:



Shared logic আলাদা module-এ move করা যায়।

54. Module Import Path

ESM-এ local file import করার সময় অনেক environment-এ extension explicit রাখা দরকার।

```
import { add } from "./math.js";
```

Node ESM-এ:

```
import { add } from "./math.js";
```

এটাই safer pattern।

55. Module Caching

Modules সাধারণত repeated import/require-এর ক্ষেত্রে একই loaded module reuse করে।

ধরো:

```
import { config } from './config.js';
```

আর অন্য file-এ:

```
import { config } from './config.js';
```

সাধারণত একই module instance-এর সাথে কাজ করা হয়।

এটা singleton-like module state তৈরি করতে পারে।

56. Module-Level State

counter.js

```
let count = 0;

export function increment() {
  count++;
}

export function getCount() {
  return count;
}
```

app.js

```
import {
  increment,
  getCount
} from './counter.js';

increment();
increment();
```

```
console.log(getCount());
```

Output:

2

Module-level state multiple consumers share করতে পারে।

57. Modules Encapsulation তৈরি করে

ধরো:

```
let balance = 1000;  
  
export function deposit(amount) {  
  balance += amount;  
}  
  
export function getBalance() {  
  return balance;  
}
```

`balance` directly অন্য module থেকে access করা যাবে না।

```
import { balance } from "../bank.js";
```

এটা কাজ করবে না যদি `balance` export করা না হয়।

এটা encapsulation-এর একটা simple example।

58. React-এ Modules

React project-এ তুমি প্রতিদিন modules ব্যবহার করো।

Example:

```
src/
├── components/
│   ├── Button.jsx
│   └── Navbar.jsx
├── utils/
│   └── formatDate.js
├── services/
│   └── api.js
└── App.jsx
```

Button:

```
export default function Button() {
  return <button>Click</button>;
}
```

App:

```
import Button from "../components/Button";
```

59. React-🔗 Named Export

```
export function Button() {
  return <button>Click</button>;
}

export function Input() {
  return <input />;
}
```

Import:

```
import {
  Button,
  Input
} from "../components";
```

60. Next.js-এ Dynamic Import

Next.js-এর মতো framework-এ dynamic loading performance optimization-এর জন্য গুরুত্বপূর্ণ।

Conceptually:

```
const HeavyComponent = dynamic(  
  () => import("./HeavyComponent")  
);
```

এখানে framework-level dynamic loading ব্যবহার করা হচ্ছে।

JavaScript-এর core concept হলো:

```
import("./HeavyComponent");
```

61. ESM-এ Top-Level `await`

ESM module-এ top-level `await` ব্যবহার করা যায় supported environments-এ।

```
const response = await fetch(  
  "https://example.com/data"  
);  
  
const data = await response.json();  
  
export default data;
```

এটা CommonJS-এর traditional model-এর সাথে একেবারে একই নয়।

62. CommonJS থেকে ESM-এ যাওয়ার সময় Common Mistakes

ভুল:

```
const math = require("./math.js");
```

ESM file-এর মধ্যে blindly `require()` ব্যবহার করা।

ESM:

```
import math from "./math.js";
```

ভুল:

```
module.exports = {  
  add  
};
```

ESM file-এ।

ESM:

```
export { add };
```

63. CommonJS বনাম ESM — Syntax Conversion

CommonJS

```
const math = require("./math");  
  
module.exports = {  
  add  
};
```

ESM

```
import { add } from "./math.js";  
  
export { add };
```

64. Default Export Conversion

CommonJS:

```
module.exports = function greet() {  
  console.log("Hello");  
};
```

ESM:

```
export default function greet() {  
  console.log("Hello");  
}
```

65. CommonJS Multiple Export Conversion

CommonJS:

```
module.exports = {  
  add,  
  subtract  
};
```

ESM:

```
export {  
  add,  
  subtract  
};
```

Import:

```
import {  
  add,  
  subtract  
} from "./math.js";
```

66. কখন ESM ব্যবহার করবে?

Modern application development-এ সাধারণত:

```
Frontend  
React  
Next.js  
Modern Node.js  
TypeScript  
Bundlers
```

এসব ক্ষেত্রে ESM জানা অত্যন্ত important।

67. কখন CommonJS দেখতে পাবে?

পুরনো বা existing Node.js project-এ:

```
const express = require("express");
```

এখনও দেখতে পারো।

অনেক legacy codebase CommonJS ব্যবহার করে।

তাই একজন JavaScript developer হিসেবে দুটোই বুঝতে হবে।

68. Common Mistake — Default vs Named Import

যদি:

```
export const user = {};
```

তাহলে:

```
import user from "./user.js";
```

ভুল।

সঠিক:

```
import { user } from "./user.js";
```

আর যদি:

```
export default user;
```

তাহলে:

```
import user from "./user.js";
```

সঠিক।

69. Common Mistake — Curly Braces

Named:

```
export const add = () => {};
```

Import:

```
import { add } from "./math.js";
```

Default:

```
export default add;
```

Import:

```
import add from "./math.js";
```

Shortcut:

```
Named    → {}  
Default → no {}
```

70. Common Mistake — Dynamic Import-এ Direct Function আশা করা

এটা ভুল ধারণা:

```
const add = await import("./math.js");  
  
add(10, 20);
```

কারণ `import()` module namespace object return করে।

সঠিক:

```
const math = await import("./math.js");  
  
math.add(10, 20);
```

Default হলে:

```
const module = await import("./math.js");  
  
module.default();
```

71. Common Mistake — `exports =`

এটা:

```
exports = {  
  add  
};
```

CommonJS export replace করার reliable way নয়।

Use:

```
module.exports = {  
  add  
};
```

72. Common Mistake — Circular Dependencies

এভাবে architecture বানানো avoid করো:

```
A → B  
↑   ↓  
└───┘
```

বরং shared logic:

```
A → shared  
B → shared
```

73. Module Design Best Practices

1. One clear responsibility

```
auth.js
```

শুধু authentication-related logic।

2. Related functionality group করো

```
math.js
```

এর মধ্যে:

```
add()  
subtract()  
multiply()  
divide()
```

3. Public API ছোট রাখো

সব internal function export করার দরকার নেই।

4. Named exports ব্যবহার করতে পারো **shared utilities**-এর জন্য

```
export function formatDate() {}  
export function formatCurrency() {}
```

5. Default export ব্যবহার করো **primary entity**-এর ক্ষেত্রে

```
export default User;
```

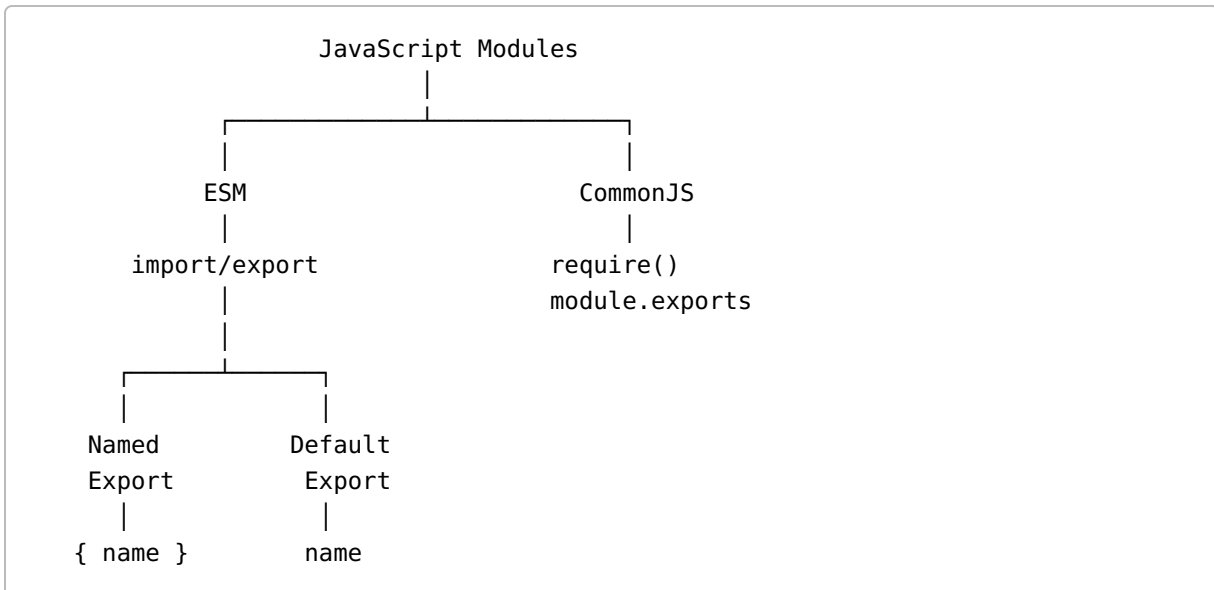
6. Circular dependency avoid করো

7. Dynamic import performance-এর প্রয়োজন অনুযায়ী ব্যবহার করো

শুধু "modern syntax" বলে সব জায়গায় dynamic import ব্যবহার করার দরকার নেই।

74. Module System — Complete Mental Model

পুরো বিষয়টা এভাবে মনে রাখো:



আর dynamic loading:

```
import()
↓
Promise
↓
Runtime module loading
↓
Lazy loading / Code splitting
```

75. Quick Comparison

ESM

```
// math.js

export function add(a, b) {
  return a + b;
}
```

```
// app.js

import { add } from "./math.js";
```

CommonJS

```
// math.js

function add(a, b) {
  return a + b;
}

module.exports = { add };
```

```
// app.js

const { add } = require("./math");
```

76. Interview Questions

Beginner

1. JavaScript module কী?
2. ES Module কী?
3. `import` এবং `export` কী?
4. Named export কী?
5. Default export কী?
6. Named এবং default export-এর difference কী?
7. `require()` কী?
8. `module.exports` কী?
9. CommonJS কী?

Intermediate

1. `exports` এবং `module.exports` এর difference কী?
2. `import()` কী return করে?
3. Dynamic import কেন ব্যবহার করা হয়?
4. Lazy loading কী?
5. Code splitting কী?
6. Static import কী?
7. Module scope কী?
8. ESM browser-এ কীভাবে কাজ করে?
9. Node.js-এ ESM enable করার উপায় কী?
10. `.mjs` এবং `.cjs` কী?
11. ESM tree shaking-এর জন্য কেন সুবিধাজনক?

Advanced

1. ESM এবং CommonJS-এর architectural difference কী?
 2. Static imports-এর সুবিধা কী?
 3. Dynamic import কীভাবে code splitting enable করে?
 4. Circular dependency কী?
 5. Module caching কী?
 6. ESM-এ default export কীভাবে represent হয়?
 7. CommonJS এবং ESM interop-এ কী ধরনের সমস্যা হতে পারে?
 8. Module-level state কীভাবে shared হতে পারে?
 9. কেন modern frontend ecosystem ESM prefer করে?
 10. CommonJS থেকে ESM migration-এর সময় কী কী বিষয় খেয়াল রাখতে হয়?
-

77. Practice — Level 1

Problem 1

`math.js` তৈরি করো:

```
add()  
subtract()  
multiply()  
divide()
```

সবগুলো named export করো।

তারপর `app.js` থেকে import করো।

Problem 2

একটি `user.js` বানাও যেখানে:

```
name  
age  
email
```

named export থাকবে।

Problem 3

একটি `User.js` বানাও যেখানে `User` class default export হবে।

তারপর অন্য file থেকে import করো।

78. Practice — Level 2

Problem 4

একটি module বানাও:

```
calculator.js
```

যেখানে:

```
export function add() {}  
export function subtract() {}  
export function multiply() {}  
export default function calculator() {}
```

তারপর একসাথে default + named imports ব্যবহার করো।

Problem 5

Dynamic import ব্যবহার করে:

```
heavyFeature.js
```

button click-এর পরে load করো।

79. Practice — Level 3

Problem 6

একটি CommonJS module বানাও:

```
math.cjs
```

যেখানে:

```
add()  
subtract()  
multiply()
```

export করবে।

তারপর অন্য `.cjs` file থেকে `require()` করবে।

Problem 7

একই functionality দুইভাবে implement করো:

```
ESM version  
CommonJS version
```

তারপর syntax compare করো।

80. Practice — Level 4

একটি ছোট project structure তৈরি করো:

```
src/  
├─ app.js  
├─ math.js  
├─ user.js  
├─ utils.js  
└─ features/  
    └─ editor.js
```

Requirements:

- `math.js` → named exports
- `user.js` → default export
- `utils.js` → multiple named exports
- `editor.js` → dynamic import

- `app.js` → সব module integrate করবে

81. Output Prediction Practice

Question 1

```
// math.js  
  
export const x = 10;
```

```
// app.js  
  
import { x } from "./math.js";  
  
console.log(x);
```

Output কী?

Question 2

```
// user.js  
  
export default "Shariar";
```

```
import name from "./user.js";  
  
console.log(name);
```

Output কী?

Question 3

```
const module = await import("./math.js");  
  
console.log(module);
```

`module` কী ধরনের value হবে?

Question 4

```
exports.name = "Shariar";  
  
exports.age = 31;
```

এখানে CommonJS-এ কী export হচ্ছে?

82. Final Quick Revision

```
Module  
↓  
Code আলাদা file-এ organize করা
```

ESM

```
export  
import
```

Named Export

```
export const name = "Shariar";
```

```
import { name } from "./user.js";
```

Default Export

```
export default function User() {}
```

```
import User from "./user.js";
```

Rename

```
import { add as sum } from "./math.js";
```

Namespace

```
import * as math from "./math.js";
```

Dynamic Import

```
const module = await import("./module.js");
```

Returns:

Promise

Useful for:

Lazy loading
Code splitting
Conditional loading
Performance optimization

CommonJS

```
module.exports = {}
```

```
const module = require("./module");
```

CommonJS shorthand

```
exports.add = add;
```

ESM vs CommonJS

ESM
→ import/export

CommonJS
→ require/module.exports

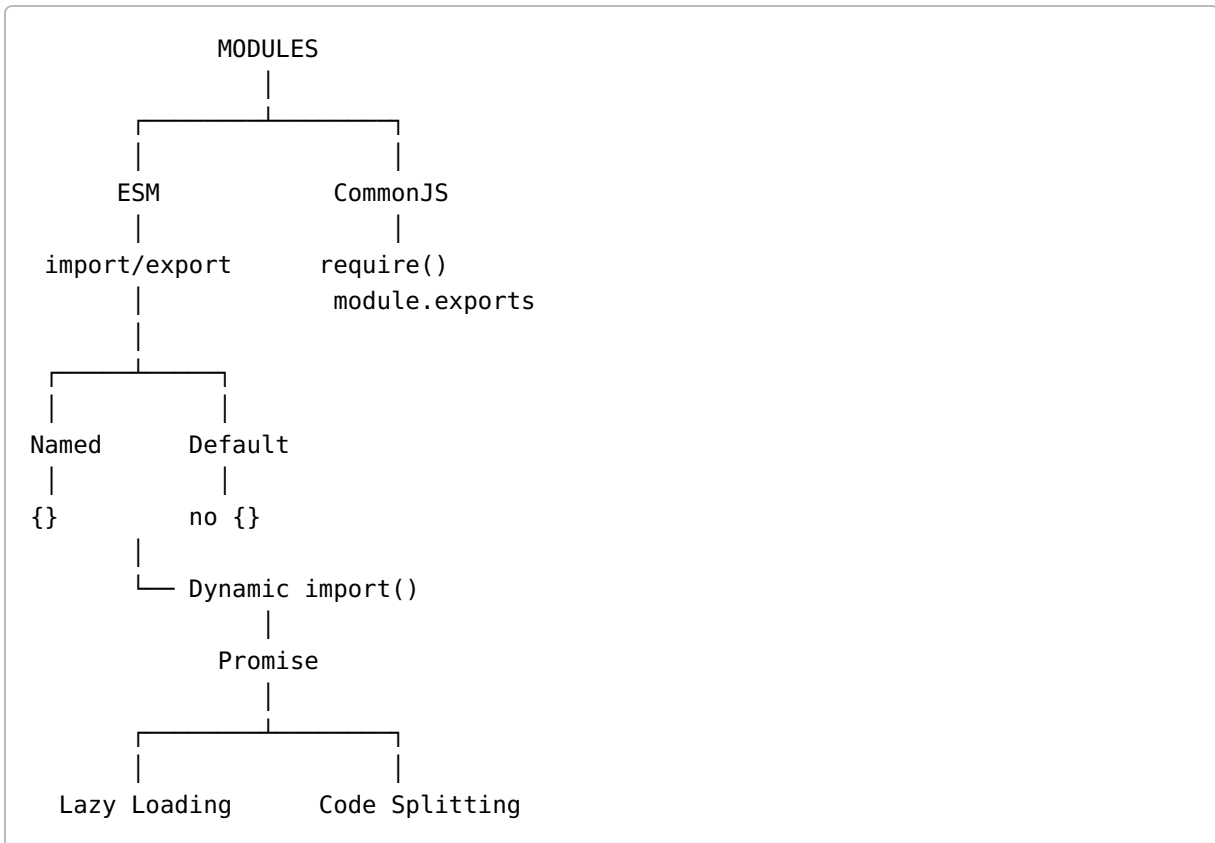
83. Mastery Checklist

এই section শেষ করার পর তুমি যেন confidently বলতে পারো:

- ☐ Module কী
- ☐ Module system কেন দরকার
- ☐ ES Modules কী
- ☐ `import` কী
- ☐ `export` কী
- ☐ Named export কী
- ☐ Default export কী
- ☐ Named vs default export difference
- ☐ `as` দিয়ে rename
- ☐ `import * as` কী
- ☐ Side-effect import কী
- ☐ Module scope কী
- ☐ Static import কী
- ☐ Dynamic `import()` কী
- ☐ Dynamic import Promise কেন return করে
- ☐ Lazy loading কী
- ☐ Code splitting কী
- ☐ Tree shaking-এর basic idea
- ☐ CommonJS কী
- ☐ `require()` কী
- ☐ `module.exports` কী
- ☐ `exports` কী
- ☐ `exports` বনাম `module.exports`
- ☐ `.mjs` কী
- ☐ `.cjs` কী
- ☐ `"type": "module"` কী
- ☐ Browser-এ ESM কীভাবে কাজ করে
- ☐ Node.js-এ ESM কীভাবে কাজ করে
- ☐ ESM vs CommonJS difference
- ☐ Circular dependency কী
- ☐ Module caching কী
- ☐ Module-level state কী
- ☐ React/Next.js-এ modules কীভাবে ব্যবহার হয়
- ☐ Dynamic import কখন ব্যবহার করতে হবে

Final Mental Model

JavaScript Modules-এর পুরো বিষয়টা মূলত এই:



সবচেয়ে গুরুত্বপূর্ণ ৫টি জিনিস

1. import/export
2. Named vs Default
3. Dynamic import()
4. require/module.exports
5. ESM vs CommonJS

এগুলো ভালোভাবে বুঝে ফেললে JavaScript-এর **Module System** অংশ practically solid হয়ে যাবে।